Lab 3 – Apache Spark + MongoDB + PostgreSQL (own work)

Laboratory goal:

The aim of the laboratory is to connect various cloud environments: AWS, Colab with a local database system hosted in a Docker container.

NOTE: !!!!

Please describe all tasks (including the code) in a *.docx document. The code and results must be thoroughly explained and described.

Tasks to be completed: → we work with various data sets: Integration of data storage in Mongodb Atlas (AWS - hosted in Hamburg) with data processing and analysis in Apache Spark (Colab) and storing the results in a relational PostgreSQL database (Docker container)

A. Learn what MongoDB is and how to set up MongoDB cloud resources on AWS. Review the *.pdf lecture material posted on the Wikamp platform in this lab.

https://www.mongodb.com/

1. Provide access to the mongoDB database engine. To do this, configure access to Mongo Atlas on Cloud

https://www.mongodb.com/cloud/atlas/register

- Insert your own cluster name: <PD_Album>
- Set the cluster's cloud provider to AWS
- Set the cluster location to Frankfurt

1.1 Set Environment Variables: Configure the environment for the Cloud:

MongoDB Atlas cluster configuration:
- enter the IP address from which the connection can be established: 0.0.0.0
- set an additional user: BIGDATA, password: BiGData

1.2 Create a new database and insert sample data into the database: Configure the environment for the Cloud:

Dataset for download: sample_mflix (movies, comments, users) or sample_analytics.
https://www.mongodb.com/docs/atlas/sample-data/sample-mflix/
or:
https://github.com/wakaidev/MFlix

B. In the MongoDB Atlas → cloud environment, define a new database for Apache Spark preprocessing:

Task 1 – Data Mining in MongoDB (40 min):

Goal: Understand document structure, calculate distributions, and write some aggregations in MongoDB. Results: Brief report.

Steps:

1. Open MongoDB Atlas UI, browse collections and sample documents.
2. Execute basic aggregate queries (number of movies per year, top genres, average rating if available).
3. Write down three research questions (e.g., "which movies have the most comments?", "what is the distribution of transactions per customer?").

Expected result: a report (1–2 sliders) with a list of document structures and 3 insights.
Assessment: accuracy of aggregation, clarity of the report.

Task 2 - Downloading Data to Spark and Preprocessing (50 min):

Goal: Connect Spark to MongoDB Atlas, load collections as DataFrames, and perform cleansing/transformation (e.g., denormalization, date field extraction, flattening).

Steps:

1. Run a **SparkSession** in Colab with a parameter that adds the MongoDB Spark Connector (example).
2. Load the **movies** and **comments** collection into a DataFrame.
3. Join (e.g., append the number of comments to the movies), filter **movies** with more than X comments, and convert dates to timestamps.
4. Prepare aggregations (top 10 genres, rating distribution/long tail).

NOTE: (If org.mongodb.spark:... does not match - adjust the version. You can also use spark.jars pointing to a local jar.)

Expected result: Spark DataFrame with cleansed data, ready for analysis.

Assessment: Working Spark→Mongo connection, correct aggregations/joins.

Task 3 – ETL: Transformations and Writing to a Relational Database (60 min):

Goal: Process and write the resulting tables to a relational database (**Postgres**). Design simple table relationships (films, film_comments_agg, genres_stats).

Table Structure (example):
FILMS (film_id PK, title, year, runtime, director)
FILM_COMMENTS_AGG (film_id FK, comments_count, avg_comment_length)
GENRES_STATS (genre, film_count, avg_comments_per_film)

Steps:

1. Install the **Docker** environment
2. Search **DockerHub** and download the Docker container with the PostgreSQL database
3. Create a JDBC connection to Postgres (parameters: url, user, password).
4. Export the DataFrame(s) to SQL tables using DataFrame.write.jdbc(...).
5. Validate the data by performing a SELECT from Postgres (you can use sqlalchemy in Colab).

Practical note: In Colab, it's easier to write to Postgres via pandas.DataFrame.to_sql() if you first convert Spark DF to Pandas (be careful with the size). For larger datasets, use JDBC.

Expected result: tables with data in the relational database, ready for SQL queries and reports.
Assessment: schema correctness, FK integrity (if used), and row completeness.

Task 4 — Results Analysis and Visualizations in Colab — 30 min:

Objective: Execute SQL queries in Postgres (or read processed DataFrames) and prepare three charts (e.g., bar, histogram, heatmap) illustrating the insights.

Prepare sample charts:

1. Top 10 films by number of comments – bar chart.
2. Distribution of the number of comments per film – histogram (log scale if very skewed).
3. Average number of comments per genre – horizontal ranking (horizontal bar).